

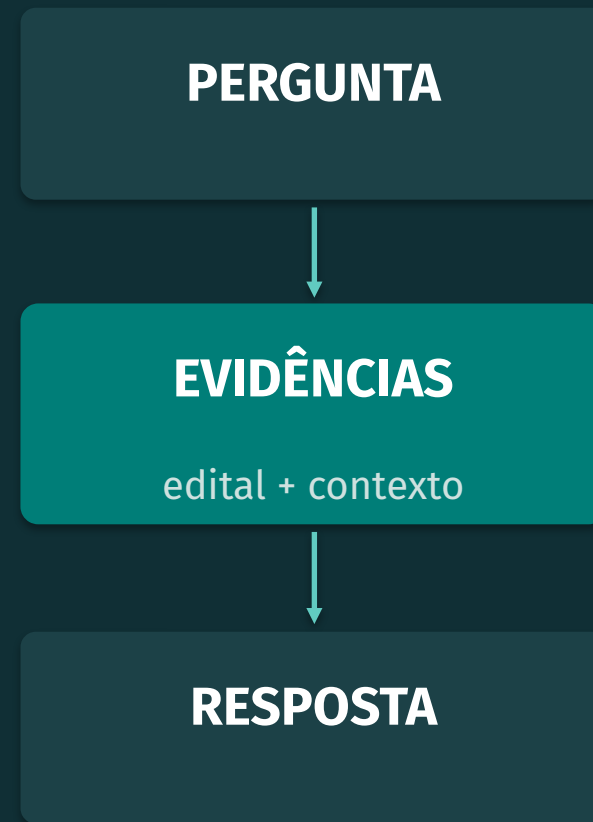
EDITE

Arquitetura de uma conversa informada

LangGraph · ferramentas · RAG

Redis · memória · operação

Guilherme Souza + Gabriel Brunetti | LEDS



Da prototipação ao uso real

Protótipo em n8n → amadurecimento em LangGraph



Dados informados pela equipe · setembro de 2026 · volumes aproximados de mensagens

Uma mensagem curta. Várias decisões por trás.

“Qual é o prazo e quais documentos preciso enviar?”

EXEMPLO DIDÁTICO · SEM RESPOSTA FICTÍCIA DO SISTEMA

**A experiência é conversacional.
O conhecimento precisa ter origem.**

Qual edital?

Qual trecho sustenta a resposta?

Qual ferramenta resolve a pergunta?

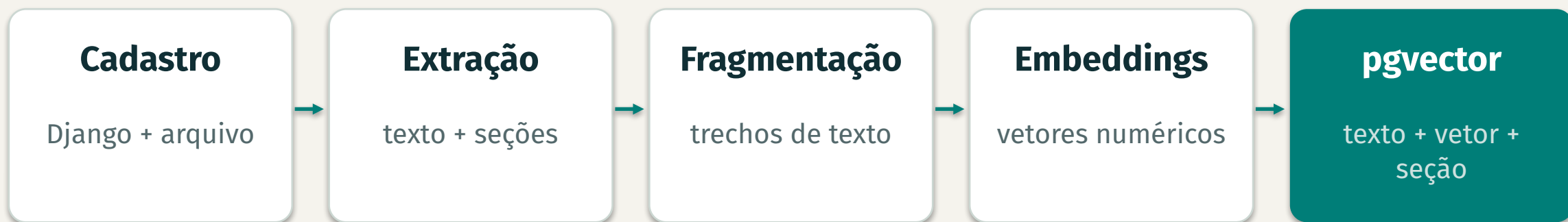
Quem conversa, quem pensa, quem mantém os dados

ATENDIMENTO · PERGUNTA E RESPOSTA ASSÍNCRONAS



Queue Monitor observa os streams. O worker Django registra mensagens e aceites em grupos próprios.

O edital vira conhecimento consultável



MinIO / S3

Arquivo original e anexos.

Catálogo no Redis

Metadados e nomes de seções.

PostgreSQL / pgvector

Trechos recuperáveis por significado.

Atualizar a base documental muda o que pode ser recuperado, sem retreinar o modelo de chat.

Primeiro localizar. Depois recuperar. Então responder.

1 Localizar o edital

Catálogo Redis → número, nome e identificação

2 Buscar o conteúdo

Embedding da pergunta → pgvector + filtros

3 Devolver evidências ao modelo

Trecho + número do edital + seção

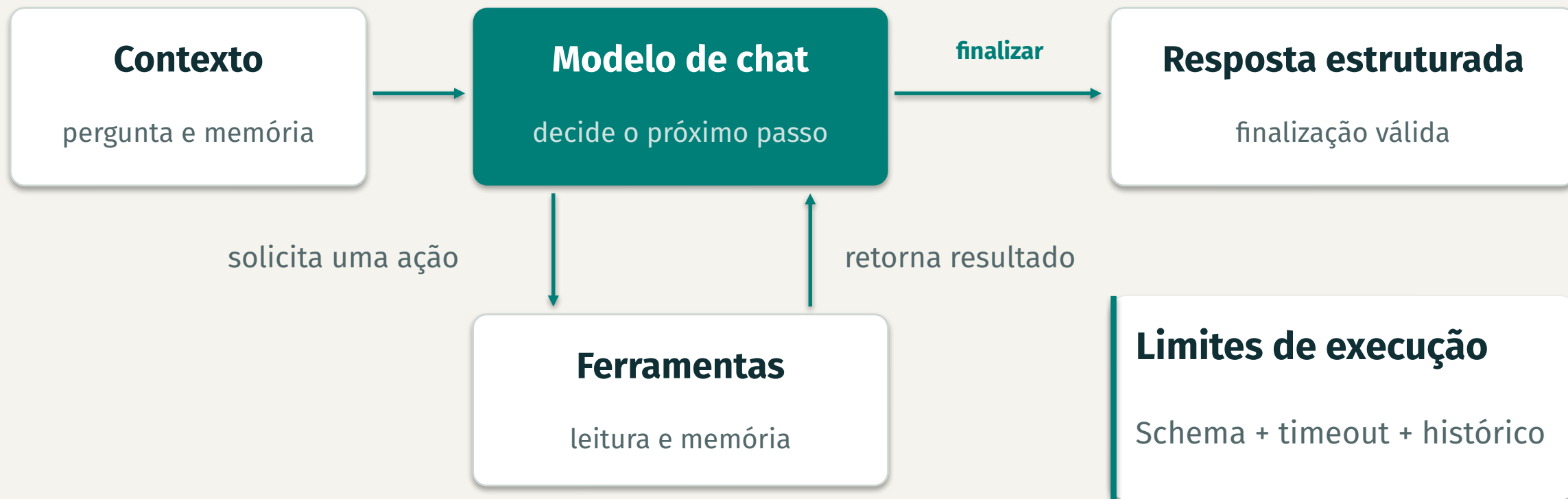
4 Compor a resposta

Contexto recuperado + pergunta + instruções

SEM RESULTADO ≠ FALHA DA BUSCA

| SIMILARIDADE ≠ VERDADE

LangGraph organiza o ciclo entre modelo e ferramentas



create_agent(...) → modelo, ferramentas e memória conectados em um grafo executável

Sete tools. Cada uma com uma responsabilidade.

CONHECIMENTO · 4 TOOLS

search_funding_calls

Localiza editais no catálogo

get_funding_call_details

Consulta os metadados de um edital

list_funding_call_sections

Descobre as seções disponíveis

search_funding_call_content

Recupera conteúdo do edital

MEMÓRIA · 3 TOOLS

remember_user_context

Guarda contexto útil do participante

remember_funding_call_relationship

Registra interesse ou vínculo com edital

forget_user_memory

Remove uma memória específica

Leitura automática no prompt dinâmico.

“Memória” não é uma coisa só

ESTADO	PARA QUÊ	ONDE
Sessão do canal	Continuidade recente	Redis · TTL de 300 s
Checkpoint	Estado atual do grafo	Redis · thread_id = session_id
Memória durável	Contexto entre sessões	PostgreSQL Store + cache Redis
Conhecimento	Evidência documental	PostgreSQL / pgvector

Contexto lembrado do usuário não substitui a versão atual do edital.

Nem toda pergunta pede busca vetorial

Filtrar atributos

“Quais editais estão abertos?”

Nome · tags · status

Catálogo no Redis

Buscar significado

“Qual é o objetivo?”

Trechos próximos da pergunta

SQL + pgvector

Ler seção inteira

“Quais são todas as datas?”

Cronograma completo

SQL filtrado por seção

A mesma tool de conteúdo oferece busca vetorial e leitura integral de seções.

Uma boa tool ensina o modelo a usá-la

`search_funding_call_content`

```
call_id      # qual edital
query        # o que procurar
sections     # onde procurar
top_k        # quantos trechos
full_sections # seção completa
```

Retorno: outcome + trechos + origem

Nome específico

A intenção fica explícita.

Descrição de uso

O agente sabe quando chamar.

Parâmetros tipados

O escopo da consulta fica claro.

Retorno com origem

A resposta ganha contexto e referência.

Do objetivo ao cronograma: muda o contexto necessário

“Qual é o objetivo deste edital?”

Localizar os trechos que explicam a finalidade.

VETORIAL

```
query="O que o edital apoia?"  
sections=[secao_objetivos]  
top_k=5  
full_sections=False
```

“Mostre o cronograma completo.”

Reunir todos os trechos da seção selecionada.

SEÇÃO INTEIRA

```
query="Cronograma completo"  
sections=[secao_cronograma]  
full_sections=True
```

Exemplos de argumentos: edital e títulos de seção vêm das tools de catálogo.

Cada peça resolve uma necessidade da aplicação de IA

TECNOLOGIA	PAPEL	POR QUE AJUDA
LLM + embeddings	Linguagem e representação	Interpretar e recuperar
LangChain / LangGraph	Tools, fluxo e estado	Compor o agente
PostgreSQL + pgvector	Filtros + busca vetorial	Escolher como consultar
Redis	Catálogo, memória e filas	Conectar canal e agente
Django + MinIO	Gestão e arquivos-fonte	Alimentar o conhecimento
Langfuse / Prometheus	Traces e métricas	Observar a execução

FastAPI + WhatsApp Cloud API levam essas capacidades até a conversa com o usuário.

Três decisões para levar ao seu agente

Contexto sob demanda

Listar e detalhar

Paginação + trechos

Controlar o contexto

Identidade no runtime

Identidade pelo aplicativo

Contexto injetado

Memória por usuário

Resposta estruturada

Texto + referências

ToolStrategy + Pydantic

Renderizar e persistir

O modelo escolhe a operação; a aplicação define contexto, escopo e formato.

O código que conecta modelo, ferramentas e memória

```
graph = create_agent(  
    model=model,  
    tools=list(tools),  
    middleware=[  
        summary,  
        DynamicPromptMiddleware(prompt_provider),  
    ],  
    response_format=ToolStrategy(StructuredAgentResponse),  
    checkpointer=checkpointer,  
    store=store,  
    context_schema=AgentRuntimeContext,  
)
```

Dependências injetadas

Modelo de chat

Tools tipadas

Checkpoint e memória

Prompt dinâmico

Agora, pense no agente da sua pesquisa

CONHECIMENTO

Dados e documentos consultáveis

FERRAMENTAS

Operações adequadas à pergunta

CONTEXTO

Fluxo e memória da conversa

Qual seria a primeira tool do agente da sua pesquisa?